AE系列篇-02.VAE理论篇: 从“重建模型”到“生成模型”

1. AE的问题与VAE的引入

1.1 AE的局限性

在上一节中，我们了解到**自编码器（AE）**的主要问题是**潜在空间是非正则化的（Unregularized）**:

1. **有限的生成能力**:只有少数区域/簇具有生成能力，整个潜在空间并不都具有生成能力
2. **空洞区域**:潜在空间中存在大量“空洞”，这些区域不对应任何训练数据
3. **随机采样无效**:从潜在空间随机采样时，很可能采样到空洞区域，生成无意义的输出

1.2 VAE的解决方案

变分自编码器（Variational Autoencoder, VAE） 解决了AE的非正则化潜在空间问题，为整个潜在空间提供了生成能力。

图1所示，VAE由两个主要部分组成:

- **编码器（Encoder）**:将高维输入数据映射到潜在空间，输出确定的潜在向量 z
- **解码器（Decoder）**:将潜在向量重构回高维数据空间

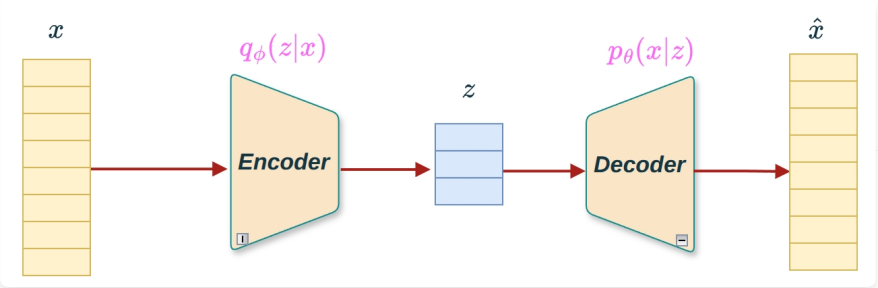


图1 VAE架构:输入 x → 编码器 $q_\phi(z|x)$ → 潜在向量 z → 解码器 $p_\theta(x|z)$ → 重构输出 $\hat{x}$

因此， 果潜在变量 z 服从某个固定的分布，换言之，基于固定的分布（ 正态分布）生成潜

变量 z，那么现在的AE便具备**生成能力**。 图2所示，VAE使用神经网络将潜变量 z 变换为观测变量 x。

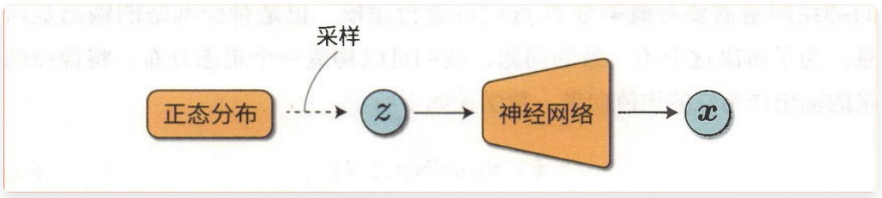


图2 VAE使用解码器将潜变量 z 变换为观测变量x

固定正态分布:VAE假设 z 是基于固定的正态分布生成的，也就是均值为 0、协方差矩阵为 I（I 为单位矩阵）的正态分布。因此，我们可以说"z 服从 $\mathcal{N}(z;0,I)$"，数学式 下所示:

$$p(z)=\mathcal{N}(z;0,I) \tag{1}$$

虽然潜变量 z 是基于简单的概率分布 $\mathcal{N}(z;0,I)$ 生成的，但是经由神经网络进行了复杂的变换。通过加深神经网络的层数并使用大量数据进行训练，可以实现更复杂的变换。由于这种神经网络负责从潜变量到观测变量的变换，因此也被称为**解码器（Decoder）**。

解码器的概率建模:我们的目标是得到观测变量 x 的概率分布 $p(x)$。因此，将 z 变换为 x 的神经网络需要对概率分布 $p(x|z)$ 进行建模。然而，神经网络的输出是向量，而不是概率分布。为了解决这个不一致的问题，我们可以构造一个正态分布，将神经网络的输出作为它的均值向量。数学式 下所示:

$$\begin{aligned}\hat{x}&=\mathrm{NeuralNet}(z;\theta)\\ p_\theta(x|z)&=\mathcal{N}(x;\hat{x},I)\end{aligned} \tag{2}$$

其中，z 表示输入，$\hat{x}$ 表示输出，θ 表示神经网络的参数。另外，假设 x 服从以 $\hat{x}$ 为均值向量的正态分布。为了便于探讨，假设协方差矩阵为单位矩阵 I。

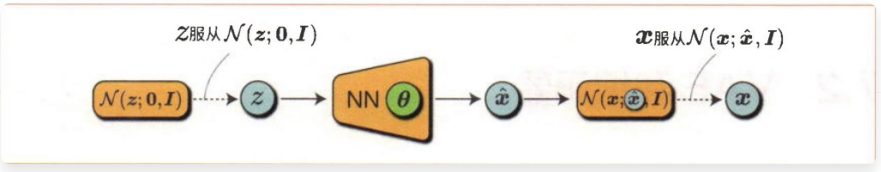


图3 VAE的解码器执行的处理（"NN"表示神经网络），虚线箭头表示"输出的随机变量服从该概率分布"，在生成数据时，可以将虚线视为采样

---

2. VAE的架构与工作原理

2.1 什么是变分自编码器?

2.1.1 变分贝叶斯视角

核心思想:我们希望将输入映射到一个**分布**，而不是固定的向量。假设我们知道该分布的真实参数 θ^*。为了生成一个看起来像真实数据点的样本 $\mathbf{x}$，我们遵循以下步骤:

1. 首先，从先验分布 $p_{\theta^*}(\mathbf{z})$ 中采样一个 $\mathbf{z}^{(i)}$
2. 然后，从条件分布 $p_{\theta^*}(\mathbf{x}|\mathbf{z}=\mathbf{z}^{(i)})$ 中生成一个值 $\mathbf{x}^{(i)}$

概率模型定义:

数据输入 $\mathbf{x}$ 和潜在编码向量 $\mathbf{z}$ 之间的关系可以完全定义为:

- **先验（Prior）**:$p_\theta(\mathbf{z})$ - 潜在变量的先验分布
- **似然（Likelihood）**:$p_\theta(\mathbf{x}|\mathbf{z})$ - 给定潜在变量生成数据的条件分布
- **后验（Posterior）**:$p_\theta(\mathbf{z}|\mathbf{x})$ - 给定数据推断潜在变量的分布

优化目标:

最优参数 θ^* 是最大化生成真实数据样本的概率:

$$\theta^* = \arg\max_\theta \prod_{i=1}^{n} p_\theta(\mathbf{x}^{(i)})$$

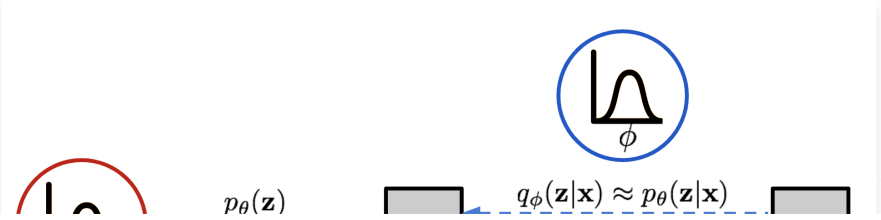
使用对数概率将乘积转换为求和:

$$\theta^* = \arg\max_\theta \sum_{i=1}^{n} \log p_\theta(\mathbf{x}^{(i)}) \tag{3}$$

为了涉及编码向量，我们更新方程为:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}|\mathbf{z}) p_\theta(\mathbf{z}) d\mathbf{z} \tag{4}$$

问题:直接计算这个积分很困难，因为需要检查所有可能的 $\mathbf{z}$ 值并求和。为了缩小值空间以加快搜索，我们引入一个新的近似函数 $q_\phi(\mathbf{z}|\mathbf{x})$，参数化为 ϕ，来输出给定输入 $\mathbf{x}$ 的可能编码，图4所示。



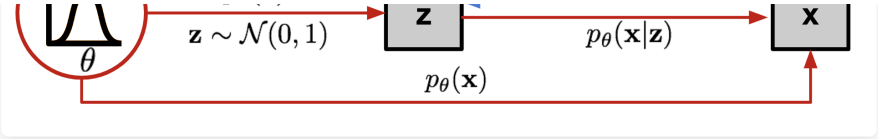


图4 VAE的图模型:实线表示生成分布 p_θ，虚线表示近似分布 q_ϕ 用于近似难以处理的后验分布 $p_\theta(\mathbf{z}|\mathbf{x})$

结构对应关系:

- 条件概率 $p_\theta(\mathbf{x}|\mathbf{z})$ 定义了生成模型，类似于解码器，也称为**概率解码器（Probabilistic Decoder）**
- 近似函数 $q_\phi(\mathbf{z}|\mathbf{x})$ 是**概率编码器（Probabilistic Encoder）**，类似于编码器

2.1.2 损失函数:ELBO

估计的后验分布 $q_\phi(\mathbf{z}|\mathbf{x})$ 应该非常接近真实的后验分布 $p_\theta(\mathbf{z}|\mathbf{x})$。我们可以使用**Kullback-Leibler（KL）散度**来量化这两个分布之间的距离。

我们希望最小化 $D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z}|\mathbf{x}))$ 关于 ϕ。

为什么使用反向KL散度?

- **前向KL散度**: $D_{\mathrm{KL}}(P\|Q)$；我们必须确保在 $P(z) > 0$ 的地方 $Q(z) > 0$。优化后的变分分布 Q 必须覆盖整个 P
- **反向KL散度**: $D_{\mathrm{KL}}(Q\|P)$；最小化反向KL散度会将 Q 压缩到 P 下（详见参考文献[1]）

在VAE中，我们使用反向KL散度，这会使估计的后验分布更加紧凑。

ELBO推导:

展开KL散度:

$$
\begin{aligned}
D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z}|\mathbf{x})) &= \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})} d\mathbf{z} \\
&= \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x}) p_\theta(\mathbf{x})}{p_\theta(\mathbf{z},\mathbf{x})} d\mathbf{z} \\
&= \int q_\phi(\mathbf{z}|\mathbf{x}) \big(\log p_\theta(\mathbf{x}) + \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z},\mathbf{x})} \big) d\mathbf{z} \\
&= \log p_\theta(\mathbf{x}) + \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z},\mathbf{x})} d\mathbf{z} \\
&= \log p_\theta(\mathbf{x}) + \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{x}|\mathbf{z}) p_\theta(\mathbf{z})} d\mathbf{z} \\
&= \log p_\theta(\mathbf{x}) + \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p_\theta(\mathbf{x}|\mathbf{z}) - \log p_\theta(\mathbf{z})] \\
&= \log p_\theta(\mathbf{x}) + D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z})) - \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z})
\end{aligned}
$$

重新排列后，我们有:

$$\log p_\theta(\mathbf{x}) = D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}|\mathbf{x})) + \mathcal{L}(\mathbf{x};\theta,\phi) \tag{5}$$

其中:

$$\mathcal{L}(\mathbf{x};\theta,\phi) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) - D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z})) \tag{6}$$

$\mathcal{L}(\mathbf{x};\theta,\phi)$ 被称为**证据下界（Evidence Lower Bound，ELBO）**。由于KL散度总是非负的，我们有:

$$\log p_\theta(\mathbf{x}) \geq \mathcal{L}(\mathbf{x};\theta,\phi) \tag{7}$$

因此，**最大化ELBO等价于最小化KL散度**。

ELBO的组成:

1. **重构项（Reconstruction Term）**:$\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z})$ - 鼓励模型从潜在编码 $\mathbf{z}$ 重构原始输入 $\mathbf{x}$
2. **正则化项（Regularization Term）**:$-D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z}))$ - 鼓励估计的后验分布 $q_\phi(\mathbf{z}|\mathbf{x})$ 接近先验分布 $p_\theta(\mathbf{z})$

损失函数:

在变分贝叶斯方法中，损失函数是ELBO的负值:

$$\mathcal{L}_{VAE} = -\mathcal{L}(\mathbf{x};\theta,\phi) = -\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) + D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p_\theta(\mathbf{z})) \tag{8}$$

通过最小化损失，我们实际上是在最大化生成真实数据样本概率的下界。

2.2 ELBO的优化

基于之前的推导，我们总结 下:

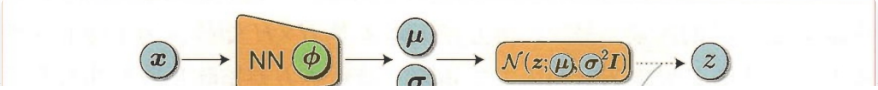
解码器（Decoder）:执行从潜变量 z 到观测变量 x 的变换，其数学式 下所示:

$$p(z) = \mathcal{N}(z;0,I)$$

$$\hat{x} = \mathrm{NeuralNet}(z;\theta)$$

$$p_\theta(x|z) = \mathcal{N}(x;\hat{x},I)$$

先验 $p(z)$ 是正态分布，似然 $p_\theta(x|z)$ 也是正态分布，容易知道，后验 $p_\theta(z|x)$ 必然为正态分布，所以编码器应该具备 下图5所示的结构:



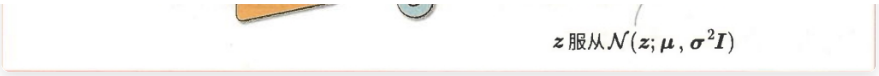


图5 编码器的结构:输入 x → 神经网络 → 输出 (μ, σ) → 后验分布 q_φ(z|x) = N(z; μ, σ²I)

编码器（Encoder）:执行从观测变量 x 到潜变量 z 的变换，其数学式　下所示:

$$\mu, \sigma = \text{NeuralNet}(x; \phi)$$

$$q_\phi(z|x) = \mathcal{N}(z; \mu, \sigma^2 I) \tag{9}$$

优化目标:我们的目标是在使用这两个神经网络的模型中，找到使对数似然 $\log p_\theta(x)$ 最大化的参数 θ 和 ϕ。

从2.1的推导，我们可以知道ELBO由两项构成。两项中的第一项可以表示为期望值，第二项可以表示为KL散度。数学式　下所示:

$$\text{ELBO}(\mathbf{x}; \theta, \phi) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{10}$$

我们将第一项定义为 J_1，第二项定义为 J_2。

2.2.1 第一项:重建误差项 J_1

首先来看 J_1 的计算方法。由于 J_1 是期望值，因此可以用蒙特卡洛方法来近似计算。具体来说，首先基于 $q_\phi(\mathbf{z}|\mathbf{x})$ 生成一些随机数，然后使用生成的随机数计算 $\log p_\theta(\mathbf{x}|\mathbf{z})$ 的平均值。即使样本量为1，VAE的效果通常也很好。因此，我们可以通过 $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$ 采样一个 $\mathbf{z}$，并通过计算 $\log p_\theta(\mathbf{x}|\mathbf{z})$ 来近似 J_1。由此，J_1 可以按　下方式计算:

$$\mu, \sigma = \text{NeuralNet}(\mathbf{x}; \phi)$$

$$\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mu, \sigma^2 I)$$

$$\hat{\mathbf{x}} = \text{NeuralNet}(\mathbf{z}; \theta)$$

$$J_1 \approx \log \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, I)$$

符号“∼”表示采样，“≈”表示近似相等。作为参考，上述计算过程　图6所示:

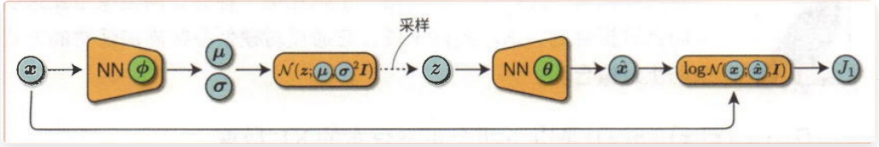


图6 J_1 的计算流程:编码器从输入数据 x 采样 z，解码器从 z 生成 $\hat{x}$，计算重建误差

这里进行的计算开始于编码器 $\text{NeuralNet}(\mathbf{x}; \phi)$ 从输入数据 $\mathbf{x}$ 中采样 $\mathbf{z}$，在此基础上，通过解码器 $\text{NeuralNet}(\mathbf{z}; \theta)$ 重新生成数据 $\hat{\mathbf{x}}$。重新生成的数据 $\hat{\mathbf{x}}$ 越接近原始输入数据 $\mathbf{x}$，

J_1 的值就越大。因此，J_1 可以被称为 **重建误差项**。

另外，$J_1 \approx \log \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, I)$ 可以进一步做　下计算:

$$\begin{aligned} J_1 &\approx \log \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, I) \\ &= -\frac{1}{2}(\mathbf{x} - \hat{\mathbf{x}})^T I^{-1}(\mathbf{x} - \hat{\mathbf{x}}) + \text{const} \\ &= -\frac{1}{2}(\mathbf{x} - \hat{\mathbf{x}})^T (\mathbf{x} - \hat{\mathbf{x}}) + \text{const} \end{aligned} \tag{11}$$

上面的计算使用了以下公式: - $|I| = 1$（单位矩阵的行列式为1） - $I^{-1} = I$（单位矩阵的逆矩阵为单位矩阵）

上面在关于 J_1 的计算中出现了不包含参数的常数项，但在优化参数时，这些常数项可以忽略。因此，为了便于理解，式子中用 const 表示常数项。

还有一个式子是　$(\mathbf{x} - \hat{\mathbf{x}})^T (\mathbf{x} - \hat{\mathbf{x}})$，它表示向量的内积。向量内积的计算可以表示为平方误差（其中 $\mathbf{x}$ 的维度为 D）:

$$(\mathbf{x} - \hat{\mathbf{x}})^T (\mathbf{x} - \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \sum_{i=1}^{D} (x_i - \hat{x}_i)^2$$

2.2.2 第二项:一致性项 J_2（KL散度）

ELBO的第二项是KL散度项。由于要最大化ELBO，因此我们希望第二项接近于0。$q_\phi(\mathbf{z}|\mathbf{x})$　越接近 $p(\mathbf{z})$，$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$ 就越接近0。因此，第二项承担着使 $q_\phi(\mathbf{z}|\mathbf{x})$ 接近 $p(\mathbf{z})$ 的使命。这一项用于判断两个概率分布的一致性（相似程度），因此可以称为**一致性项**。

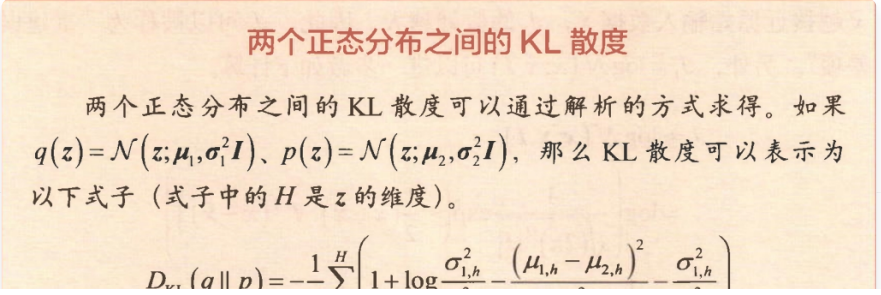
一致性项具有**正则化（Regularization）**的作用。**正则化（Regularization）**是一种通过调整模型复杂度来防止过拟合的方法。具体来说，它通过对模型参数施加惩罚的方式来防止过拟合。

$D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$ 是以下两个正态分布的KL散度:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu, \sigma^2 I)$$

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; 0, I)$$

两个正态分布之间的KL散度　图7所示:


两个正态分布之间的 KL 散度

两个正态分布之间的 KL 散度可以通过解析的方式求得。如果 $q(z)=\mathcal{N}\left(z;\boldsymbol{\mu}_1,\boldsymbol{\sigma}_1^2\boldsymbol{I}\right)$、$p(z)=\mathcal{N}\left(z;\boldsymbol{\mu}_2,\boldsymbol{\sigma}_2^2\boldsymbol{I}\right)$，那么 KL 散度可以表示为以下式子（式子中的 H 是 z 的维度）。

$$D_{KL}(q\|p)=-\frac{1}{2}\sum^{H}\left(1+\log\frac{\sigma_{1,h}^2}{\quad}-\frac{\left(\mu_{1,h}-\mu_{2,h}\right)^2}{\quad}-\frac{\sigma_{1,h}^2}{\quad}\right)$$

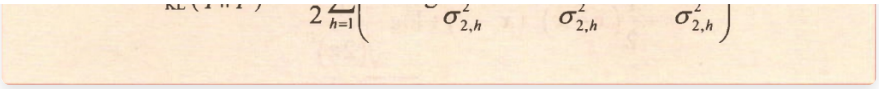


图7 两个正态分布之间的KL散度计算

因此，这里的KL散度可以做　下计算:

$$J_2 = D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) = \frac{1}{2}\sum_{i=1}^{H}[\mu_i^2 + \sigma_i^2 - 1 - \log\sigma_i^2] \tag{12}$$

2.2.3 ELBO的完整计算

基于以上探讨，ELBO可以做　下计算:

$$\mathrm{ELBO}(\mathbf{x};\theta,\phi) = -\frac{1}{2}\|\mathbf{x}-\hat{\mathbf{x}}\|^2 - \frac{1}{2}\sum_{i=1}^{H}[\mu_i^2 + \sigma_i^2 - 1 - \log\sigma_i^2] + \mathrm{const} \tag{13}$$

作为参考，图8展示了计算ELBO的流程。图8所示的编码器和解码器模型让人联想起“古老”的自编码器，这也是VAE被称为**变分自编码器（Variational Autoencoder）**的原因。

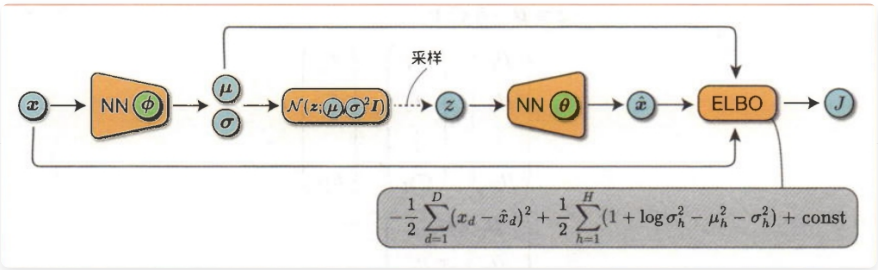


图8 计算ELBO的流程:编码器从输入数据x采样z，解码器从z生成x̂，ELBO = J_1 - J_2

2.3 重参数化技巧（Reparameterization Trick）

为了最大化ELBO，我们需要更新VAE的参数 θ 和 ϕ。由于这两个参数都是神经网络的参数，可以通过梯度下降法进行优化。与EM算法不同（EM算法需要固定一个参数并更新另一个参数），VAE可以同时更新 θ 和 ϕ，前提是能够计算ELBO关于这两个参数的梯度。

然而，梯度计算面临一个关键挑战。　图8所示，VAE的计算图包含采样操作。**采样操作是不可微分的**，这意味着梯度无法通过采样节点反向传播，从而阻碍了端到端的训练。要解决这个问题，需要引入**重参数化技巧（Reparameterization Trick）**。

2.3.1 为什么需要重参数化?

在VAE中，我们需要从近似后验分布 $q_\phi(z|x) = \mathcal{N}(\mu, \sigma^2)$ 中采样潜在向量 z。然而，直接采样 $z \sim \mathcal{N}(\mu, \sigma^2)$ 会导致梯度无法通过采样操作传播回编码器，因为采样是一个随机过程，

不具有可微性。

核心问题:采样操作阻断了梯度流，使得无法使用反向传播算法优化编码器参数 ϕ。

2.3.2 重参数化解决方案

重参数化技巧（Reparameterization Trick）通过将随机采样转换为可微分的确定性计算来解决这个问题:

$$z = \mu + \sigma \odot \epsilon \tag{14}$$

其中: - μ 和 σ 是编码器的输出（可微分） - $\epsilon \sim \mathcal{N}(0, I)$ 是从标准正态分布采样的随机噪声（独立于模型参数） - $\odot$ 表示逐元素相乘

要点: - **随机性分离**:随机性被“移出”计算图，转移到独立于模型参数的随机变量 ϵ - **梯度可传播**:梯度可以通过 μ 和 σ 反向传播，因为它们是编码器的确定性输出 - **可微分采样**:采样过程从不可微分的随机操作转变为可微分的确定性计算

图9所示:

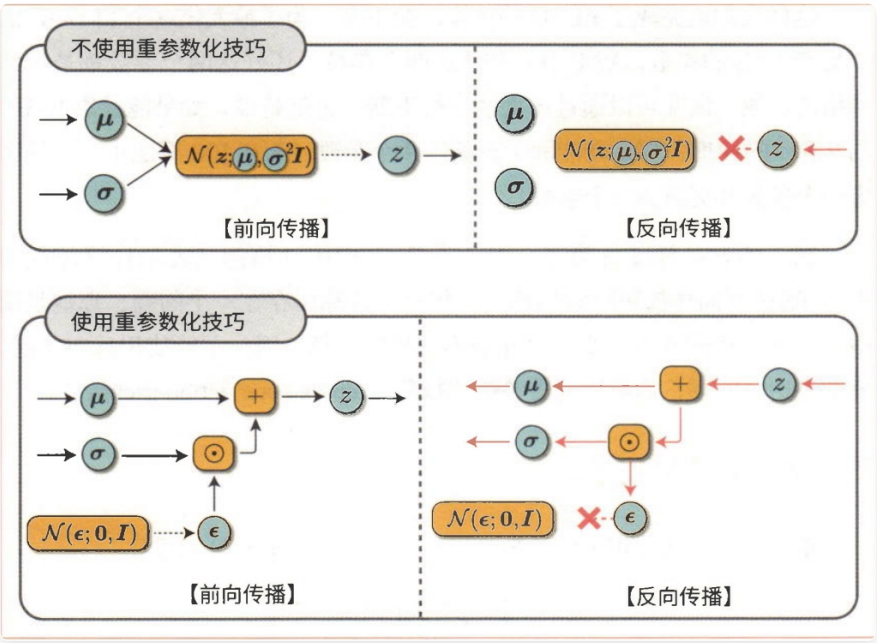


图9 重参数化技巧:将不可微分的采样操作转换为可微分的确定性计算

---

3. VAE概述

变分自编码器（Variational Autoencoder, VAE） 是一种生成模型，它通过正则化潜在空间来解决AE的生成问题。

VAE的核心思想:

- **正则化潜在空间**:对潜在空间施加分布约束，使其接近标准正态分布
- **编码器输出分布参数**:编码器不再直接输出潜在向量，而是输出分布的均值 μ 和标准差 σ
- **重参数化技巧**:通过重参数化使采样过程可微分，支持反向传播

VAE的结构　图10所示:

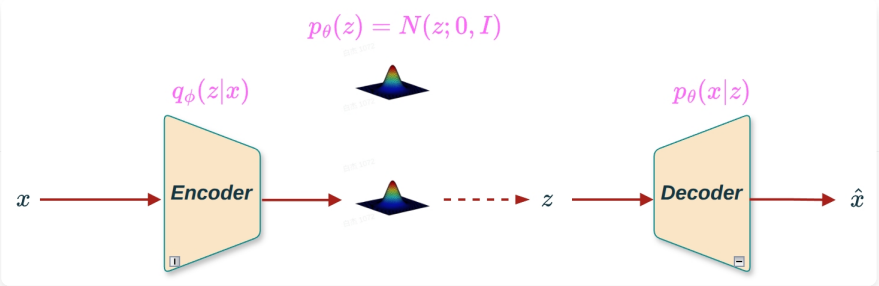


图10 VAE结构:编码器输出分布参数 (μ, σ)，通过重参数化采样 z，解码器重构输出 $\hat{x}$

图10展示了VAE的基本结构和工作流程。为了更全面地理解VAE的架构，图11进一步展示了VAE的完整架构示意图，包括编码器、解码器以及它们之间的连接关系。

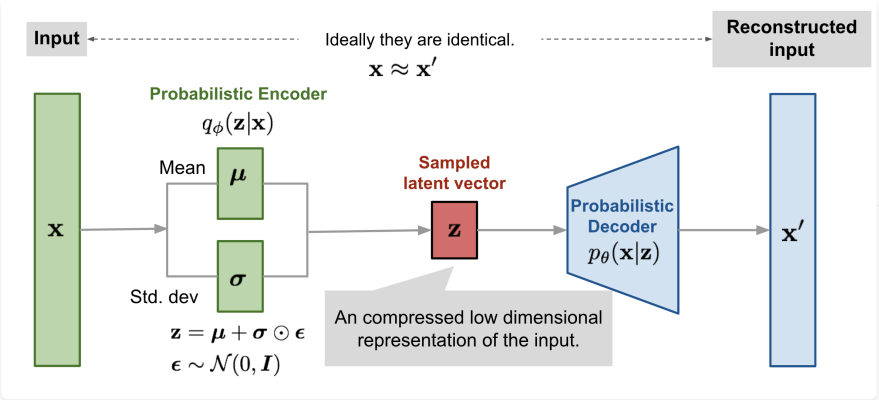


图11 VAE的完整架构示意图:展示了编码器、解码器以及潜在空间的完整结构

---

4. 什么是“正则化”?

4.1 正则化与非正则化的概念

正则化（Regularization）是指对某个量施加约束或限制，而**非正则化（Unregularized）**意味着没有约束，可以是任意值。

在AE中，编码器直接输出一个确定的潜在向量 z，这个 z 可以是任意值，没有分布约束。训练

时，编码器只学习了训练数据对应的那些 z 值，导致潜在空间中只有训练数据对应的区域是“有效的”，其他区域是“空洞”（gaps），没有对应任何训练数据。 果从空洞区域采样，解码器会生成无意义的输出。

形象地说，把潜在空间想象成一张地图:**AE**的地图上只有几个标记点（训练数据对应的位置），其他地方是空白/未知区域；而**VAE**的整张地图都被“填满”，每个位置都有意义，可以安全地访问任何地方。

下图展示了在MNIST数据集上训练网络后生成的潜在空间示例。图12展示了**AE + KL正则化**（即VAE）的潜在空间，图13展示了**AE没有KL正则化**的潜在空间。

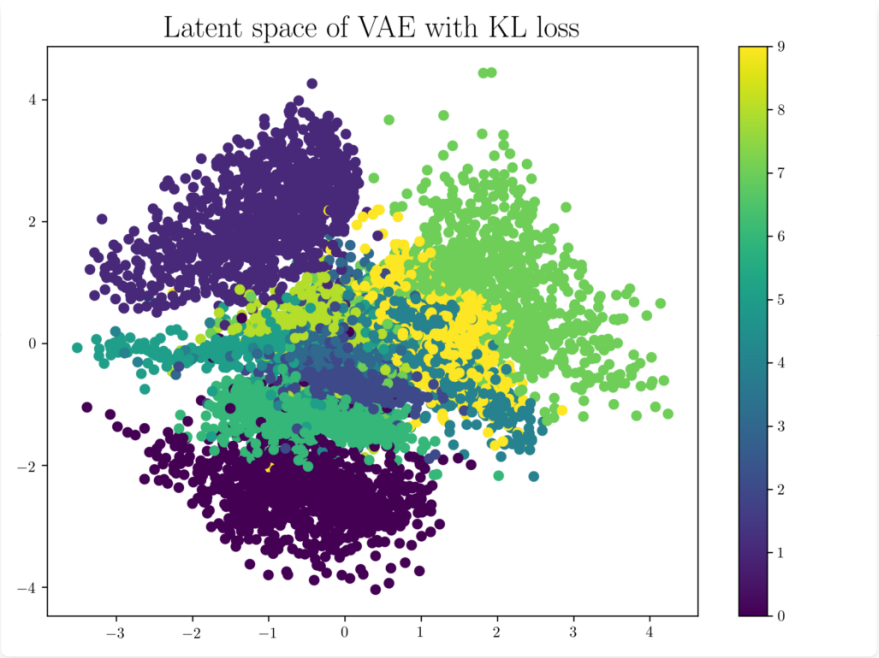


图12 AE + KL正则化（VAE）的潜在空间可视化:潜在空间被正则化为接近标准正态分布，簇之间没有明显的空洞

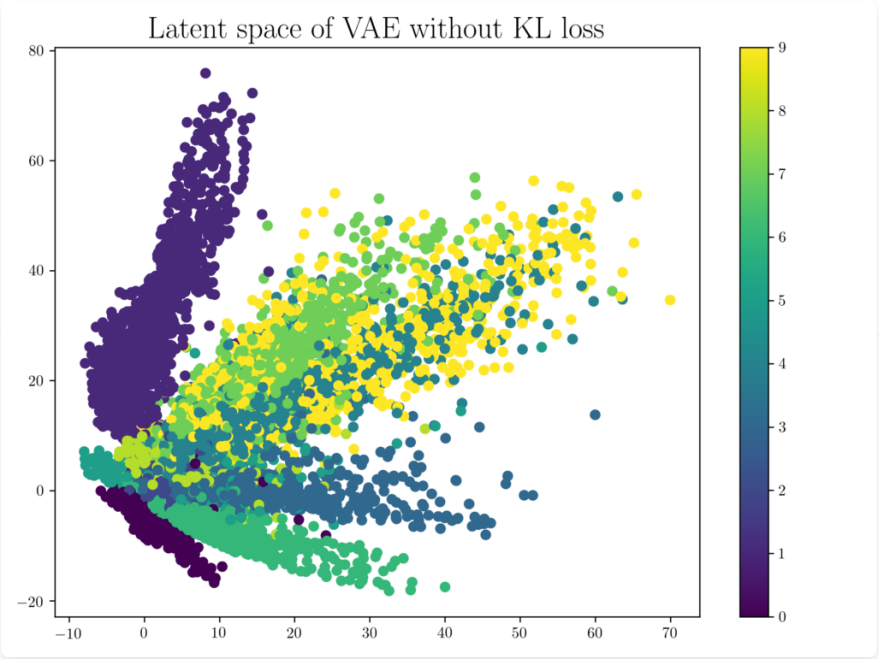


图13 AE没有KL正则化的潜在空间可视化:潜在空间存在明显的空洞区域

从图12和图13的对比中可以发现:

AE + KL正则化（VAE）的训练过程试图在两种损失（重构损失和KL散度损失）之间找到平衡，最终得到一个类似于单位范数的潜在空间分布，其中相似的输入数据点被聚类在一起。单位范数条件确保潜在空间均匀分布，且聚类之间没有明显的间隙。实际上，相似数据输入的聚类通常会在某些区域重叠，聚类之间没有间隙，并且该空间类似于单位范数分布。

而**AE没有KL正则化**的潜在空间存在明显的“空洞”（gaps），只有训练数据对应的区域是有效的，从空洞区域采样会生成无意义的输出。

从实际影响来看，**非正则化（AE）**中，从 $z = [1.2, -0.5, 3.1, \ldots]$ 采样可能生成有意义的图像（ 果这个区域有训练数据），但从 $z = [10.0, 10.0, 10.0, \ldots]$ 采样可能生成无意义的图像（空洞区域）。而**正则化（VAE）**中，从标准正态分布 $\mathcal{N}(0, I)$ 采样任何 z 都能生成有意义的图像（因为整个空间都被正则化了）。

4.2 数学上的区别

AE（非正则化）: - 编码器: $z = \text{encoder}(x)$ → 输出确定的值 - 潜在空间:可以是任意分布，没有约束 - 结果:潜在空间不连续，存在空洞

VAE（正则化）: - 编码器: $(\mu, \sigma) = \text{encoder}(x)$ → 输出分布参数 - 潜在空间:被约束为接近标准正态分布 $\mathcal{N}(0, I)$ - 结果:潜在空间连续、平滑，没有空洞

4.3 为什么叫“正则化”?

在VAE中，KL散度损失项:

$$\text{KL}(q_\phi(z|x) || p(z)) = \text{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, I))$$

这个损失项: - 强制 $q_\phi(z|x)$ 接近标准正态分布 $\mathcal{N}(0, I)$ - 这就是对潜在空间的“正则化”（施加分布约束） - 没有这个约束 = 非正则化

总结:非正则化意味着潜在空间没有分布约束，导致存在“空洞”区域，这些区域不对应任何训练数据，因此无法生成有意义的输出。正则化通过约束潜在空间分布（ 接近标准正态分布）来消除这些空洞，使整个空间都具有生成能力。

参考文献

---

1. From Autoencoder to Beta-VAE↩